SCHOOL OF TECHNOLOGY	

DATABASE MANAGEMENT SYSTEMS

PROJECT-BASED LEARNING (PBL) REPORT

Small Database Application Development using Java / Python

PROJECT TITLE

SMART PARKING LOT ALLOCATION AND BILLING SYSTEM

Program	**B-TECH**
Section	**AIML-RHINOS**
Faculty	**Prof.Sahithi**
Date of Submission	**15-10-2026**
Academic Year	**2026 – 27**

Submitted by

Roll Number	Student Name	Mobile	Email ID
25WU0102066	**D. Snigdha Choudary**	**8074718408**	**snigdha.d_2029@woxsen.edu.in**

ACKNOWLEDGEMENT

I express my sincere gratitude to Woxsen University, School of Technology, for providing me with the opportunity to undertake this Project-Based Learning (PBL) activity as part of the Database Management Systems (DBMS) course.

I thank my course faculty, Prof.Sahithi mam, for the guidance, suggestions, and support throughout the design and development of this project. The guidance helped me understand the practical application of database concepts such as ER modelling, relational schema design, normalization, SQL, constraints, queries, and database application development.

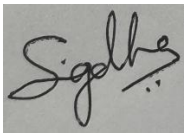
This PBL activity has strengthened my understanding of DBMS concepts and provided valuable experience in applying them to a real-world problem.

UNDERTAKING

I hereby declare that the project entitled “Smart Parking Lot Allocation and Billing System,” submitted as part of the Project-Based Learning activity for the Database Management Systems course, represents the work prepared for academic evaluation.

This report is based on the supplied project source code, SQL scripts, documentation, test results, and application screenshots. Implementation details and test outcomes are described to the extent supported by those materials. Any external technical concepts used in the report are acknowledged through references.

I undertake that I am responsible for the correctness and authenticity of the work submitted and that I have followed the academic and project guidelines prescribed by the University.

Signature: 

Name: D. Snigdha Choudary

Roll Number: **25WU0102066**

Date: **15-10-2026**

ABSTRACT

The Smart Parking Lot Allocation and Billing System is a database-driven desktop application designed to manage parking facilities, spaces, customers, vehicles, reservations, parking sessions, passes, tariffs, bills, and payments. Manual or disconnected records can make it difficult to identify available spaces, prevent conflicting reservations, track vehicles entering and leaving, and calculate charges consistently.

The objective of this project is to implement a centralized relational database and connect it to a Python desktop interface for operational parking management. The system uses MySQL 8.0 and Python with Tkinter, with MySQL Connector/Python providing database connectivity.

The database contains normalized entities linked through primary and foreign keys, supported by unique and check constraints, triggers, functions, stored procedures, and reporting views. Application features include free-slot search, reservation handling, gate entry and exit, automatic slot allocation, duration-based billing, part payments, parking passes, tariff management, customer and vehicle maintenance, and operational reports. Validation is performed in the application and reinforced by database rules.

The supplied project documentation records 23 automated tests passing and 20 database constraint tests designed to reject invalid operations. The project demonstrates how a relational database can support consistent parking workflows, enforce business rules, and produce useful occupancy and revenue information.

Keywords: Smart Parking, MySQL, Python Tkinter, Relational Database, Normalization, Billing and Reporting.

TABLE OF CONTENTS

S. No.	Contents	**Page No.**
1	Acknowledgement	2
2	Undertaking	3
3	Abstract	4
4	List of Acronyms	9
5	Chapter 1: Problem Statement, Solution Strategy and ER Diagram	10
5.1	Problem Statement	10
5.2	Objectives	10
5.3	Scope and Assumptions	10
5.4	Proposed Solution Strategy	11
5.5	Functional Requirements	12
5.6	Entity-Relationship Diagram (ERD)	12
5.7	Entity and Relationship Summary	13
6	Chapter 2: Logical Database Design	14
6.1	Relational Schema	14
6.2	Keys and Integrity Constraints	15
6.3	Functional Dependencies	16
6.4	Normalization up to Third Normal Form (3NF)	17
6.5	Data Dictionary	18
6.6	Implementation of Logical Design: DDL and Sample Data	19
7	Chapter 3: Application Development Using Logical Database Design	20
7.1	Application Overview	20
7.2	Development Environment and Database Connectivity	20
7.3	Application Modules and Database Mapping	21

7.4	CRUD Operations	22
7.5	Business Rules, Validation and Error Handling	23
7.6	Reports and Database Operations	24
7.7	Application Screens	25
7.8	Testing and Results	31
8	Chapter 4: Conclusion	32
8.1	Limitations and Future Enhancements	33
9	Chapter 5: References	34
10	Chapter 6: Appendices	35
10.1	Appendix A — Complete Database Script	35
10.2	Appendix B — Application Source Code	35
10.3	Appendix C — Additional Test Data and Outputs	36

2. List of Figures

Figure No.	Figure Title	**Page No.**
Figure 1	Entity-Relationship Diagram of the Smart Parking Lot Allocation and Billing System	13
Figure 2	Dashboard	25
Figure 3	Free Slot Search	26
Figure 4	Gate Entry and Slot Allocation	26
Figure 5	Gate Exit and Bill Generation	27
Figure 6	Billing and Part Payment	27
Figure 7	Reservation Management	28
Figure 8	Customer Record Added	28
Figure 9	Peak-Hour Report	29
Figure 10	Violations Report	29
Figure 11	Revenue Report	30
Figure 12	Occupancy Report	30

List of Tables

Table No.	Table Name	Description
Table 1	facility	Stores parking facility details.
Table 2	floor	Stores floors within each parking facility.
Table 3	zone	Stores zones within each floor.
Table 4	vehicle_type	Stores supported vehicle categories.
Table 5	slot	Stores parking slot details and availability status.
Table 6	tariff	Stores parking rates and daily caps.
Table 7	customer	Stores customer information.
Table 8	vehicle	Stores vehicle registration details and owner information.
Table 9	reservation	Stores parking reservations and their status.
Table 10	pass	Stores parking passes, validity periods, and fees.
Table 11	parking_session	Records vehicle entry, exit, and parking duration.
Table 12	bill	Stores parking charges and billing status.
Table 13	payment	Records payment amounts, methods, and dates.

LIST OF ACRONYMS

Acronym	Expansion
CRUD	Create, Read, Update, Delete
DBMS	Database Management System
ERD	Entity-Relationship Diagram
FK	Foreign Key
GUI	Graphical User Interface
PK	Primary Key
SQL	Structured Query Language
3NF	Third Normal Form
UI	User Interface

CHAPTER 1: PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM

1.1 Problem Statement

Parking facilities must coordinate the physical location and type of each parking slot with vehicle ownership, reservations, active parking sessions, tariffs, passes, and payments. When this information is maintained manually or in separate records, staff may struggle to find suitable free spaces, detect overlapping reservations, identify vehicles already inside, calculate parking charges, and track outstanding balances.

The Smart Parking Lot Allocation and Billing System addresses these needs through a centralized MySQL database and a Python Tkinter desktop application. The application searches for compatible slots, allocates spaces at gate entry, records exit time, calculates bills from parking duration and tariff rules, accepts full or partial payments, manages passes and reservations, and produces occupancy, peak-hour, violation, and revenue reports.

1.2 Objectives

The main objectives of the project are:

1. To design a relational schema for facilities, floors, zones, vehicle types, slots, customers, vehicles, reservations, passes, parking sessions, tariffs, bills, and payments.
2. To implement primary keys, foreign keys, unique constraints, check constraints, triggers, functions, and stored procedures to enforce important integrity and operational rules.
3. To develop a Python desktop application using Tkinter and MySQL Connector/Python to interact with the database.
4. To support customer and vehicle CRUD operations, free-slot search, reservation management, gate entry and exit, billing, part payments, passes, and tariff maintenance.
5. To generate database-backed reports and verify workflows through automated tests, constraint tests, and documented application screenshots.

1.3 Scope and Assumptions

The scope of the system includes the management of parking facilities, parking-slot availability, customer and vehicle records, reservations, parking sessions, passes, tariffs, bills, and payments.

Area	Scope or assumption

Facility structure	Facilities contain floors; floors contain zones; zones contain slots.
Slot allocation	Each slot is associated with a vehicle type and has AVAILABLE, OCCUPIED, or MAINTENANCE status.
Customer and vehicle records	Customers own vehicles, and vehicle registration numbers are unique and validated.
Reservations	A vehicle can reserve a compatible slot for a time window, subject to availability and overlap checks.
Parking sessions	Entry and exit are recorded, and the system allocates slots while preventing conflicting active sessions.
Tariffs and bills	Tariffs vary by facility and vehicle type and may include a daily cap. Bill amounts are calculated from session duration.
Passes and payments	Passes have validity periods and fees; bills can be paid in full or through part payments.
Reports	The application includes occupancy, free-slot, peak-hour, longest-stay, pass-usage, violation, and revenue reports.
Out of scope	External payment-gateway integration, cloud deployment, and a production multi-site authentication system are not documented as implemented.
Assumption	The application is an academic prototype running with a local MySQL service.

1.4 Proposed Solution Strategy

The proposed solution integrates a relational database with a desktop application. MySQL stores the structured information, while Python and Tkinter provide the user interface through which parking staff interact with the system.

The main workflow is:

Customer and vehicle registration → Optional reservation or pass → Gate entry and slot allocation → Parking session → Gate exit and bill generation → Payment processing → Reports.

The solution follows these steps:

1. Database design: Identify the entities, relationships, attributes, primary keys, and foreign keys.
2. Schema implementation: Create the database tables and integrity constraints using SQL.

3. Business-rule implementation: Use database triggers, stored procedures, and functions to handle important parking operations.
4. Application development: Build the desktop interface and connect it to MySQL through MySQL Connector/Python.
5. Validation: Check user input in the application and enforce important rules in the database.
6. Reporting: Use database queries and views to display occupancy, parking durations, pass usage, violations, and revenue.
7. Testing: Use the supplied automated tests, constraint tests, and application screenshots to document the behaviour of the system.

1.5 Functional Requirements

Req. ID	Functional requirement	Priority
FR-01	The system shall maintain customer records with validated names, phone numbers, and email addresses.	High
FR-02	The system shall register, search, update, and remove vehicle records while preventing duplicate registration numbers.	High
FR-03	The system shall search available slots by facility and vehicle type.	High
FR-04	The system shall create, list, cancel, and expire reservations while checking time windows and slot conflicts.	High
FR-05	The system shall record gate entry and automatically allocate a compatible slot.	High
FR-06	The system shall record exit time and calculate parking charges using applicable tariffs and daily caps.	High
FR-07	The system shall record full and partial payments and calculate outstanding balances.	High
FR-08	The system shall manage parking passes and tariff records.	Medium
FR-09	The system shall display dashboard information and database-backed reports.	Medium
FR-10	The system shall reject invalid inputs and enforce key business rules at both application and database levels.	High

1.6 Entity-Relationship Diagram (ERD)

The ER diagram represents the major entities in the Smart Parking Lot Allocation and Billing System and the relationships among them.

The database schema contains 13 operational tables. The facility hierarchy is represented through facilities, floors, zones, and slots. Customers are associated with vehicles and passes. Vehicles can have reservations and parking sessions. Parking sessions are associated with bills, and bills can have multiple payment records.

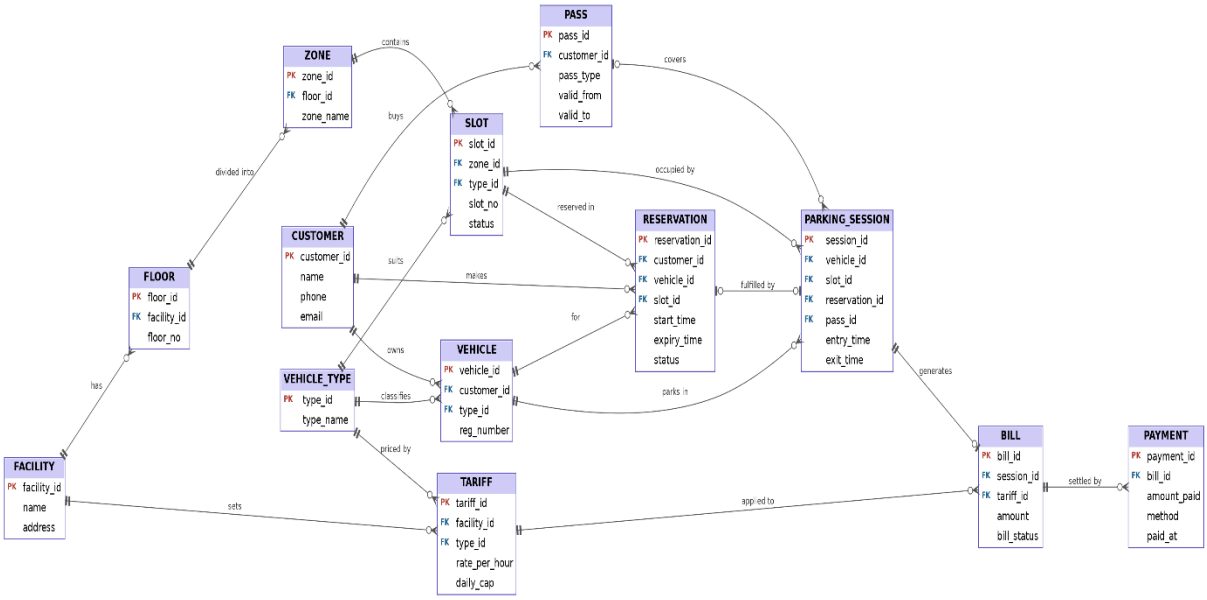


Figure 1. Entity-Relationship Diagram of the Smart Parking Lot Allocation and Billing System

Insert the supplied project image docs/smart_parking_er_diagram.png here.

The implemented database schema should be treated as the authoritative design if the ER diagram is revised further.

1.7 Entity and Relationship Summary

Entity or relationship	Key attributes or cardinality	Purpose or rule
Facility	facility_id; one-to-many with floor	Stores facility name and address.
Floor	floor_id, facility_id, floor_no	Represents levels within a facility.
Zone	zone_id, floor_id, zone_name	Groups slots within a floor.

Vehicle Type	type_id, type_name	Defines supported vehicle categories.
Slot	slot_id, zone_id, type_id, slot_no, status	Represents a physical parking space.
Tariff	tariff_id, facility_id, type_id, effective_from, rate_per_hour, daily_cap	Defines charges by facility, vehicle type, and effective date.
Customer and Vehicle	One customer to many vehicles	A customer may own multiple vehicles.
Vehicle and Reservation	One vehicle to many reservations over time	Stores booking windows and status.
Customer and Pass	One customer to many passes	Stores purchased parking passes and validity.
Vehicle/Slot and Parking Session	Multiple sessions over time	Records entry and optional exit for a parking stay.
Parking Session and Bill	One session to zero or one bill	A completed session may have a generated bill.
Bill and Payment	One bill to many payments	Allows multiple payments against a bill, supporting partial payment.

CHAPTER 2: LOGICAL DATABASE DESIGN

2.1 Relational Schema

The ER model is represented by 13 MySQL tables. Primary keys uniquely identify records, and foreign keys connect related entities. The SQL schema also defines indexes and controlled relationships to preserve referential integrity.

The 13 operational tables are:

S. No.	Table name	Primary key	Purpose
1	facility	facility_id	Parking facility master data
2	floor	floor_id	Facility floor information

3	zone	zone_id	Named zone on a floor
4	vehicle_type	type_id	Supported vehicle categories
5	slot	slot_id	Individual parking spaces and statuses
6	tariff	tariff_id	Rates by facility, vehicle type, and effective date
7	customer	customer_id	Customer contact details
8	vehicle	vehicle_id	Vehicle registration and owner/type references
9	reservation	reservation_id	Reserved slot and time window
10	pass	pass_id	Customer pass type, validity, and fee
11	parking_session	session_id	Vehicle parking entry and exit
12	bill	bill_id	Calculated charge for a parking session
13	payment	payment_id	Payment transactions against bills

The tables are connected through primary-key and foreign-key relationships. This structure reduces unnecessary duplication and makes it possible to retrieve related information using SQL joins.

2.2 Keys and Integrity Constraints

The database uses primary keys, foreign keys, unique constraints, check constraints, enumerated values, triggers, functions, and stored procedures to maintain data accuracy.

Entity or object	**Key or constraint**	**Rule enforced**
All tables	Primary key	Each row has a unique identifier.
floor, zone, slot	Foreign keys	Connects the facility hierarchy and slot type to valid parent records.
customer	Unique phone/email where defined	Prevents duplicate contact identifiers.
vehicle	Unique reg_number; foreign keys to customer and vehicle type	Prevents duplicate registrations and invalid references.
reservation	Foreign keys and time checks	Helps prevent invalid reservation windows and conflicting use.

parking_session	Foreign keys to vehicle, slot, reservation, and pass	Keeps session references valid.
tariff	Unique facility/type/effective-date combination; positive rate check	Prevents duplicate tariff effective dates and non-positive rates.
bill	Unique session_id; foreign keys	Limits a session to one bill and ensures valid references.
payment	Foreign key to bill; positive amount check; method validation	Prevents invalid payment references and values.
slot.status	Enumerated values and default	Restricts slot state to AVAILABLE, OCCUPIED, or MAINTENANCE.
Triggers and procedures	Session, reservation, billing, and payment checks	Enforces slot availability, compatibility, billing, payment, and state-transition rules.

The schema contains eight triggers, two SQL functions, and five stored procedures. The application uses parameterized SQL statements, while the database independently checks important business rules.

2.3 Functional Dependencies

Functional dependencies describe how one attribute or group of attributes determines other attributes in a relation.

Relation	**Important functional dependencies**
facility	facility_id → name, address; name → facility_id, address
floor	floor_id → facility_id, floor_no; (facility_id, floor_no) → floor_id
zone	zone_id → floor_id, zone_name; (floor_id, zone_name) → zone_id
vehicle_type	type_id → type_name; type_name → type_id
slot	slot_id → zone_id, type_id, slot_no, status; (zone_id, slot_no) → slot_id
tariff	tariff_id → facility_id, type_id, effective_from, rate_per_hour, daily_cap
customer	customer_id → name, phone, email; unique phone/email values determine the customer where enforced
vehicle	vehicle_id → customer_id, type_id, reg_number; reg_number → vehicle_id

reservation	reservation_id → vehicle_id, slot_id, start_time, expiry_time, status
pass	pass_id → customer_id, pass_type, valid_from, valid_to, fee
parking_session	session_id → vehicle_id, slot_id, reservation_id, pass_id, entry_time, exit_time
bill	bill_id → session_id, tariff_id, amount, bill_status, generated_at; session_id → bill_id
payment	payment_id → bill_id, amount_paid, method, paid_at

These dependencies help establish appropriate primary keys, unique constraints, and relationships between tables.

2.4 Normalization up to Third Normal Form (3NF)

Normalization organizes data to reduce unnecessary duplication and prevent insertion, update, and deletion anomalies.

The supplied normalization note records an important design change: reservation.customer_id was removed because a reservation already identifies a vehicle, and each vehicle identifies its owner. Keeping customer_id in reservation would introduce the transitive dependency:

\[\text{reservation_id} \rightarrow \text{vehicle_id} \rightarrow \text{customer_id} \]

Customer details are therefore reached through the vehicle relation.

First Normal Form (1NF)

Each field stores a single value. Multiple payment methods and amounts are represented as separate payment rows rather than repeating values in a single cell.

Second Normal Form (2NF)

Non-key attributes depend on the whole key. Session, bill, and payment facts are separated into their own relations, avoiding partial dependencies in a combined transaction record.

Third Normal Form (3NF)

Non-key attributes depend on the key rather than other non-key attributes. Facility, floor, zone, vehicle type, slot, tariff, customer, vehicle, reservation, pass, session, bill, and payment facts are separated into their own tables.

Relation group	1NF	2NF	3NF	Reason
Facility, floor, zone	Satisfied	Satisfied	Satisfied	Facility hierarchy facts are separated by entity.

Customer, vehicle, vehicle type	Satisfied	Satisfied	Satisfied	Owner, vehicle, and vehicle-category data are stored separately.
Slot, tariff	Satisfied	Satisfied	Satisfied	Slot status/location and tariff rules are represented independently.
Reservation, pass	Satisfied	Satisfied	Satisfied	Booking windows and pass validity are independent facts.
Parking session, bill, payment	Satisfied	Satisfied	Satisfied	Session, calculated charge, and individual payments are separate.

Controlled Redundancy

slot.status supports fast availability lookups, while bill.amount and bill_status preserve the calculated historical charge and current payment state. Database triggers maintain these values so that they are not intended to be entered as arbitrary values manually.

2.5 Data Dictionary

The following summarizes the principal attributes and data types used in the implemented schema. Refer to sql/01_schema.sql for the complete definitions and constraints.

Table	Important attributes	Data types and rules
facility	facility_id, name, address	Integer primary key; name and address are required.
floor	floor_id, facility_id, floor_no	Integer identifiers; linked to a facility.
zone	zone_id, floor_id, zone_name	Integer identifiers; linked to a floor.
vehicle_type	type_id, type_name	Small integer identifier and required type name.
slot	slot_id, zone_id, type_id, slot_no, status	Identifiers, slot number, and enumerated status.
tariff	tariff_id, facility_id, type_id, effective_from, rate_per_hour, daily_cap	Identifiers, date, decimal hourly rate, optional daily cap.
customer	customer_id, name, phone, email	Integer identifier, name, 10-character phone field, email.

vehicle	vehicle_id, customer_id, type_id, reg_number	Identifiers and registration number; linked to owner and type.
reservation	reservation_id, vehicle_id, slot_id, start_time, expiry_time, status	Identifiers, date-time fields, and enumerated status.
pass	pass_id, customer_id, pass_type, valid_from, valid_to, fee	Identifiers, enumerated pass type, dates, and fee.
parking_session	session_id, vehicle_id, slot_id, reservation_id, pass_id, entry_time, exit_time	Identifiers and timestamps; exit time and optional references may be null.
bill	bill_id, session_id, tariff_id, amount, bill_status, generated_at	Identifiers, decimal amount, enumerated bill status, timestamp.
payment	payment_id, bill_id, amount_paid, method, paid_at	Identifiers, decimal payment amount, enumerated payment method, timestamp.

Important enumerated values include:

- Slot status: AVAILABLE, OCCUPIED, MAINTENANCE
- Reservation status: ACTIVE, USED, EXPIRED, CANCELLED
- Pass type: MONTHLY, QUARTERLY, ANNUAL
- Bill status: UNPAID, PARTIAL, PAID
- Payment method: CASH, CARD, UPI, WALLET

2.6 Implementation of Logical Design: DDL and Sample Data

The logical database design is implemented through SQL Data Definition Language (DDL) statements. The following is an abridged example from the facility table:

```
CREATE TABLE facility (
    facility_id INT UNSIGNED NOT NULL AUTO_INCREMENT,
    name VARCHAR(80) NOT NULL,
    address VARCHAR(150) NOT NULL,
    CONSTRAINT pk_facility PRIMARY KEY (facility_id),
    CONSTRAINT uq_facility_name UNIQUE (name)
) ENGINE=InnoDB;
```

The complete database implementation is supplied in the project archive through the following SQL scripts:

- sql/00_create_app_user.sql — creates the application database account.
- sql/01_schema.sql — creates the smart_parking database, tables, constraints, indexes, triggers, functions, and stored procedures.
- sql/02_sample_data.sql — inserts demonstration data.
- sql/03_views.sql — creates nine reporting and operational views.
- sql/04_queries.sql — contains representative joins, subqueries, aggregations, window-function queries, and reporting queries.
- sql/05_constraint_tests.sql — contains 20 deliberate invalid-operation tests.
- sql/06_prototype_demo.sql — provides a scripted demonstration of entry, exit, billing, part payment, and occupancy.

These files are retained in the submitted project ZIP rather than duplicated in full in this report, keeping the main report readable and avoiding divergence between the report and executable SQL.

CHAPTER 3: APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN

3.1 Application Overview

The Smart Parking application is a Python desktop GUI built with Tkinter. It communicates with the MySQL database through a database-access layer and service functions.

The interface has pages for the dashboard, slot search, gate operations, reservations, customers, vehicles, passes, tariffs, billing, and reports. The service layer issues SQL queries and writes records to the database for operational workflows.

The main features of the application are:

- Dashboard for current parking activity.
- Free-slot search based on facility and vehicle type.
- Vehicle entry and automatic slot allocation.
- Vehicle exit and bill generation.
- Reservation creation and management.
- Customer and vehicle record management.

- Parking pass and tariff management.
- Full and partial payment processing.
- Occupancy, peak-hour, violation, and revenue reports.

3.2 Development Environment and Database Connectivity

The application uses the following technologies.

Item	Details
Programming language	Python 3.9 or newer, as specified in the project README
GUI toolkit	Tkinter with themed reusable widgets
DBMS	MySQL 8.0.16 or newer
Connector	MySQL Connector/Python
Database name	smart_parking
Main entry point	main.py
Database layer	app/db.py
Business logic	app/services.py
Input validation	app/validators.py
Reports	app/reports.py
Interface styling	app/theme.py and app/widgets.py

Database Connectivity Flow

The database connectivity flow is:

Tkinter screen → ParkingService → Database wrapper → MySQL database → Query results → Service logic → Screen or report output

The database wrapper provides methods such as query, query_one, execute, call, and close.

SQL calls use parameter placeholders for user-supplied values. Database errors are converted into user-friendly ParkingError messages by the application error layer.

Table: Database Implementation Environment

Item	Details
DBMS	MySQL 8.0+
Database	smart_parking

Operational tables	13
Reporting views	9
Stored procedures	5
SQL functions	2
Triggers	8
Sample data	Demonstration facilities, customers, vehicles, slots, reservations, passes, sessions, bills, tariffs, and payments

3.3 Application Modules and Database Mapping

The application modules map user actions to the database tables and views.

Module or screen	**Purpose**	**Tables/views**	**Main operations**
Dashboard	Show active sessions, free/occupied slots, unpaid balance, and violations	parking_session, slot, v_free_slots, v_bill_details, v_violations	Read and aggregate
Slot Search	Find available slots by facility and vehicle type	slot, zone, floor, facility, vehicle_type, v_free_slots	Read and filter
Gate	Vehicle entry, slot allocation, exit, and bill generation	vehicle, parking_session, reservation, pass, slot, tariff, bill	Create, update, procedure calls
Reservations	Create, list, cancel, and expire bookings	reservation, vehicle, slot, facility, parking_session	Create, read, update
Customers	Search and maintain customer information	customer, vehicle	CRUD
Vehicles	Search and maintain vehicle registrations	vehicle, customer, vehicle_type	CRUD
Passes	Buy, renew, list, and delete parking passes	pass, customer, v_pass_usage	CRUD and read
Tariffs	Maintain rates and daily caps	tariff, facility, vehicle_type	Create, read, update

Billing	View bills, record payments, and track balances	bill, payment, parking_session, tariff, v_bill_details	Read and create payment
Reports	Occupancy, peak hours, durations, pass usage, violations, and revenue	Reporting views and related tables	Read, aggregate, export CSV

3.4 CRUD Operations

CRUD refers to Create, Read, Update, and Delete operations. These operations allow the application to maintain data and retrieve information as required.

Operation	Application feature	Tables affected	Evidence
Create	Add customer or vehicle; create reservation; enter vehicle; buy pass; add tariff; record payment	customer, vehicle, reservation, parking_session, pass, tariff, payment	Screenshots, service methods, and SQL procedures
Read/Search	Dashboard, customer/vehicle search, free-slot search, active sessions, bills, reservations, reports	Master and transaction tables; reporting views	Application screenshots and report queries
Update	Edit customer/vehicle, change slot status, cancel reservation, update tariff, renew pass	customer, vehicle, slot, reservation, tariff, pass	Service methods in app/services.py
Delete/Controlled removal	Delete customers, vehicles, or passes when no blocking references exist	customer, vehicle, pass	Database referential integrity and application error handling

Deletion is controlled by database relationships. Records that are referenced by other records may not be removable until the relevant relationships are resolved.

3.5 Business Rules, Validation and Error Handling

The project implements validation at both the application and database levels. The application checks user input before performing operations, while MySQL constraints and stored routines provide additional protection.

Rule or validation	Implementation	Expected behaviour
Customer name, phone, email	Python validators and database constraints where defined	Malformed or duplicate details are rejected.
Vehicle registration	Format normalization, regular-expression validation, and unique database key	Registration is normalized to uppercase; invalid or duplicate numbers are refused.
Slot compatibility and occupancy	Database triggers/procedures and service-layer allocation	A vehicle cannot occupy a slot of the wrong type or a slot already in use.
Reservation window	Date-time parsing, expiry-after-start, maximum 24-hour window, and conflict checks	Invalid or conflicting bookings are rejected.
Entry and exit state	Stored procedures and session triggers	Duplicate entry and repeated exit are refused; exit cannot precede entry.
Billing	fn_pick_tariff and fn_bill_amount, with bill triggers/procedures	Charges are derived from elapsed time and the applicable tariff/daily cap.
Payments	Positive amount validation, supported payment method, procedure/trigger validation	Overpayment is refused; partial payment updates the balance/status.
Database failures	ParkingError and friendly error conversion	Database errors are shown as readable messages rather than raw SQL failures.

The SQL project defines eight triggers, two functions, and five stored procedures. The database wrapper uses parameterized queries and catches MySQL Connector/Python errors. The supplied automated tests also cover rollback when billing fails because a suitable tariff is unavailable.

3.6 Reports and Database Operations

The application includes database-backed reports that help parking staff monitor the facility, understand usage patterns, and track payments.

Report or operation	SQL/database concept	Output

Occupancy by floor	Aggregation over slots and location views	Total, occupied, available, and maintenance slots; occupancy percentage
Free slots now	Filtering and GROUP BY	Free-space counts by facility and vehicle type
Peak hours	Time extraction and ranking	Entry counts and busy-hour ranking
Longest stays	Timestamp difference and ordering	Longest active or completed parking durations
Pass usage	Joins and aggregation	Pass state, visits, hours parked, and fee
Violations	UNION ALL and date/time rules	Overstay, expired-pass, long-stay, and unpaid-bill flags
Revenue by day	GROUP BY date and facility	Payments collected per day and facility
Revenue by facility/method	Joins, aggregation, and window functions	Billed, collected, outstanding, payment counts, and payment-method share
Unpaid bills/bill details	View and calculated balance	Bill amount, paid amount, balance, tariff, and status

The project contains nine reporting views:

1. v_slot_location
2. v_occupancy
3. v_free_slots
4. v_session_duration
5. v_peak_hours
6. v_pass_usage
7. v_violations
8. v_bill_details
9. v_revenue_daily

These views support operational queries and reporting by presenting information derived from the underlying database tables.

3.7 Application Screens

Figure 2. Dashboard

The dashboard summarizes current parking activity, free and occupied spaces, unpaid balances, and violations.

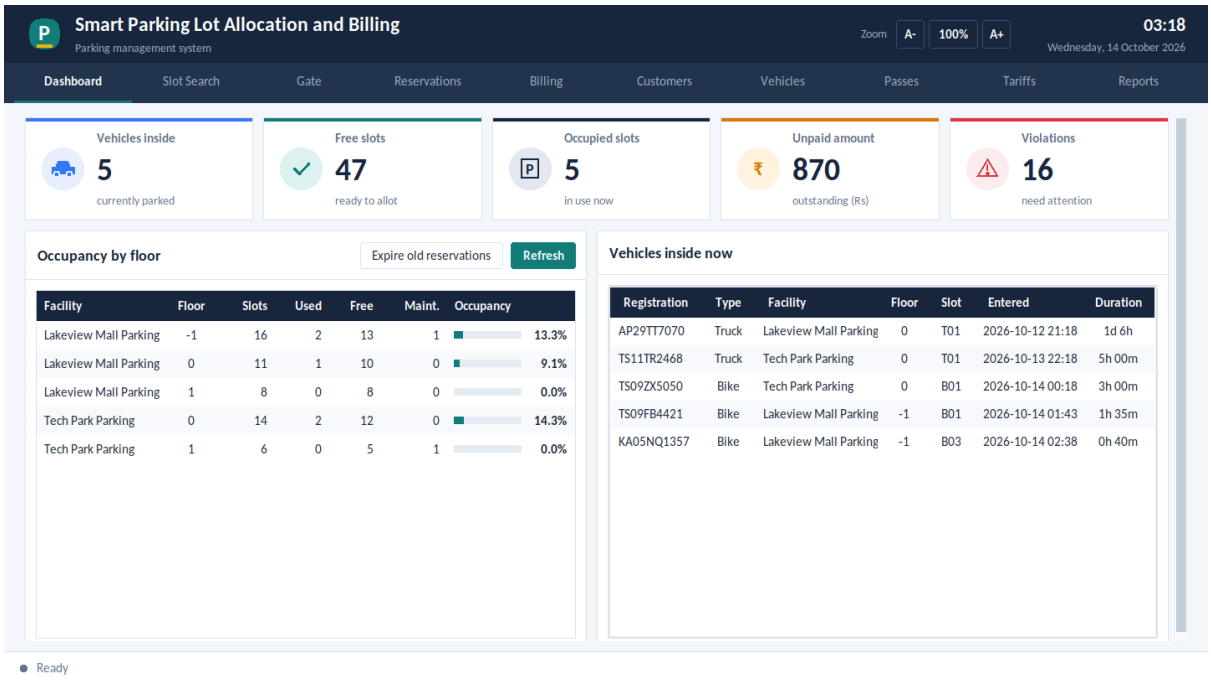


Figure 3. Free Slot Search

The slot search screen filters available spaces by facility and vehicle type.

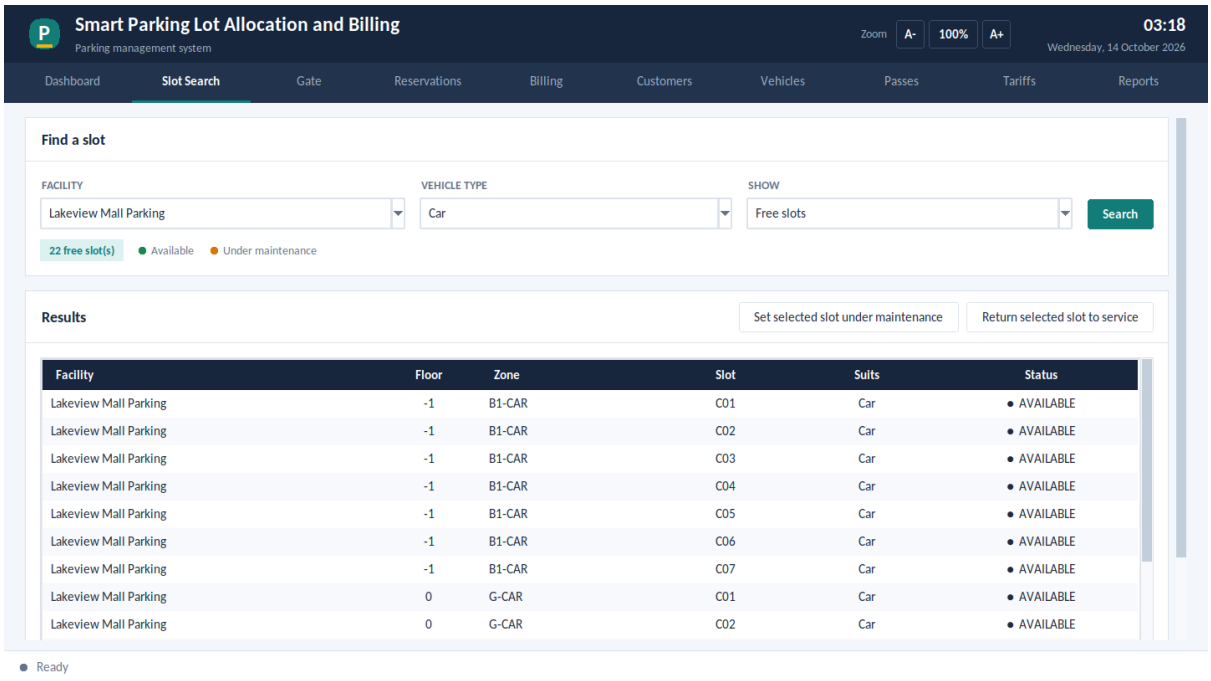


Figure 4. Gate Entry and Slot Allocation

The gate workflow identifies a registered vehicle and allocates a compatible available slot.

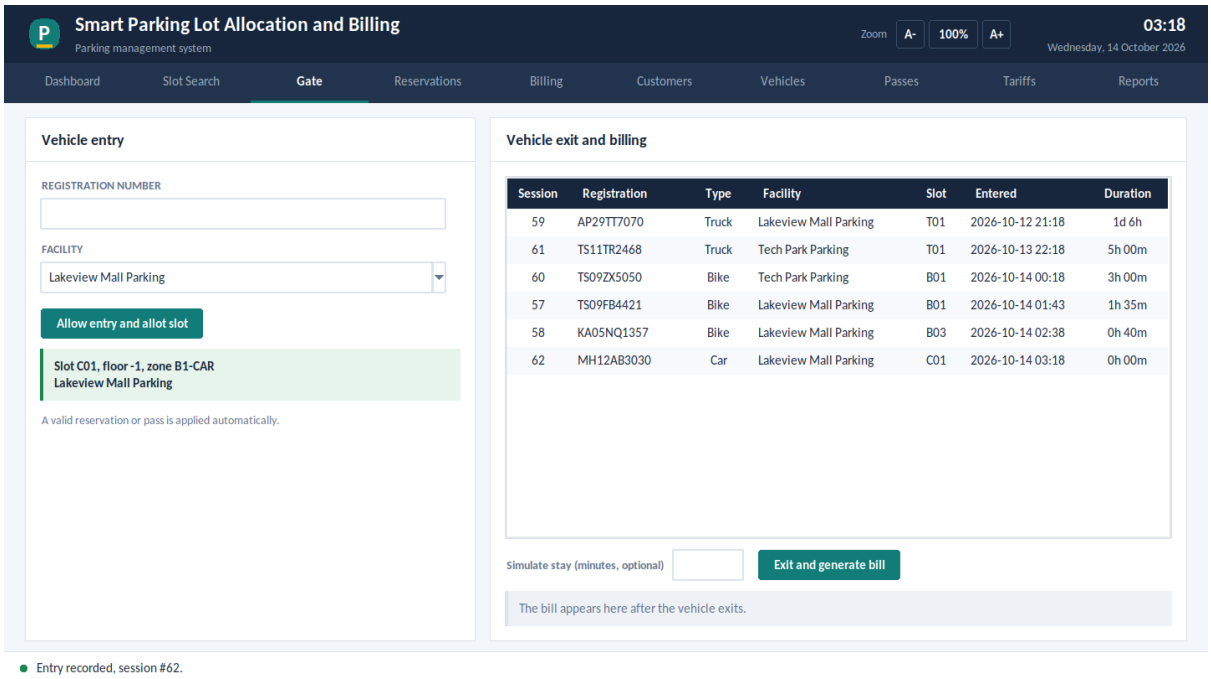


Figure 5. Gate Exit and Bill Generation

The exit workflow closes a parking session and displays the calculated bill.

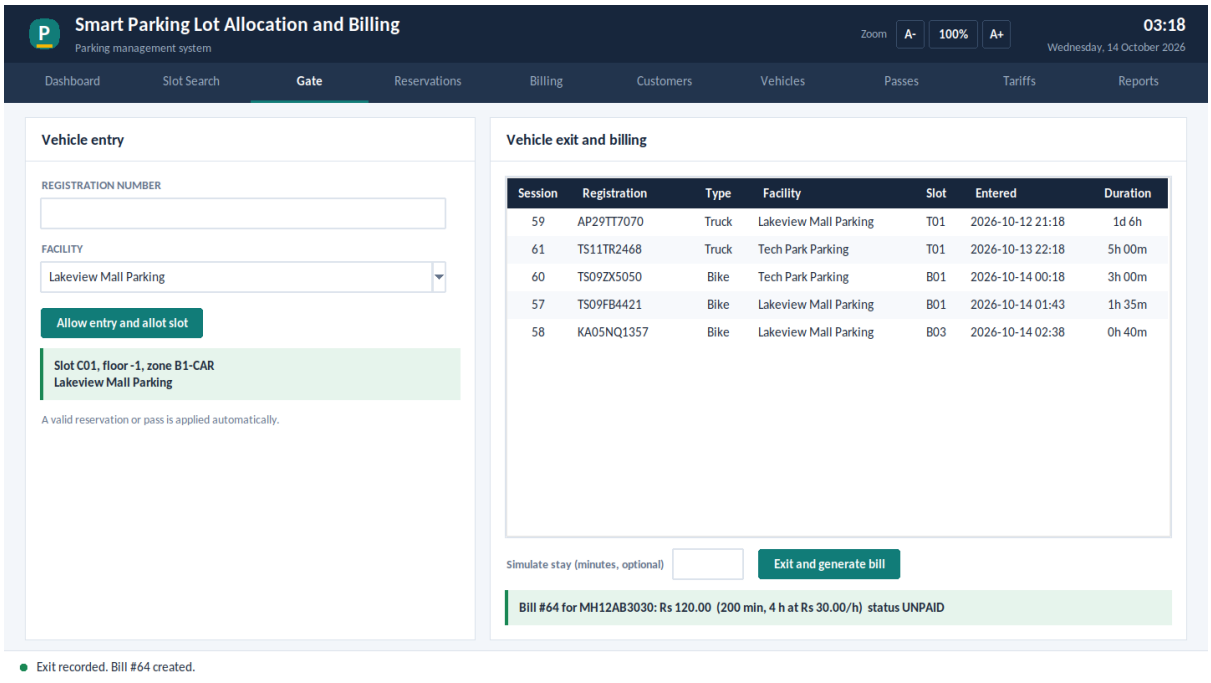


Figure 6. Billing and Part Payment

The billing screen records a partial payment and shows the remaining balance and bill status.

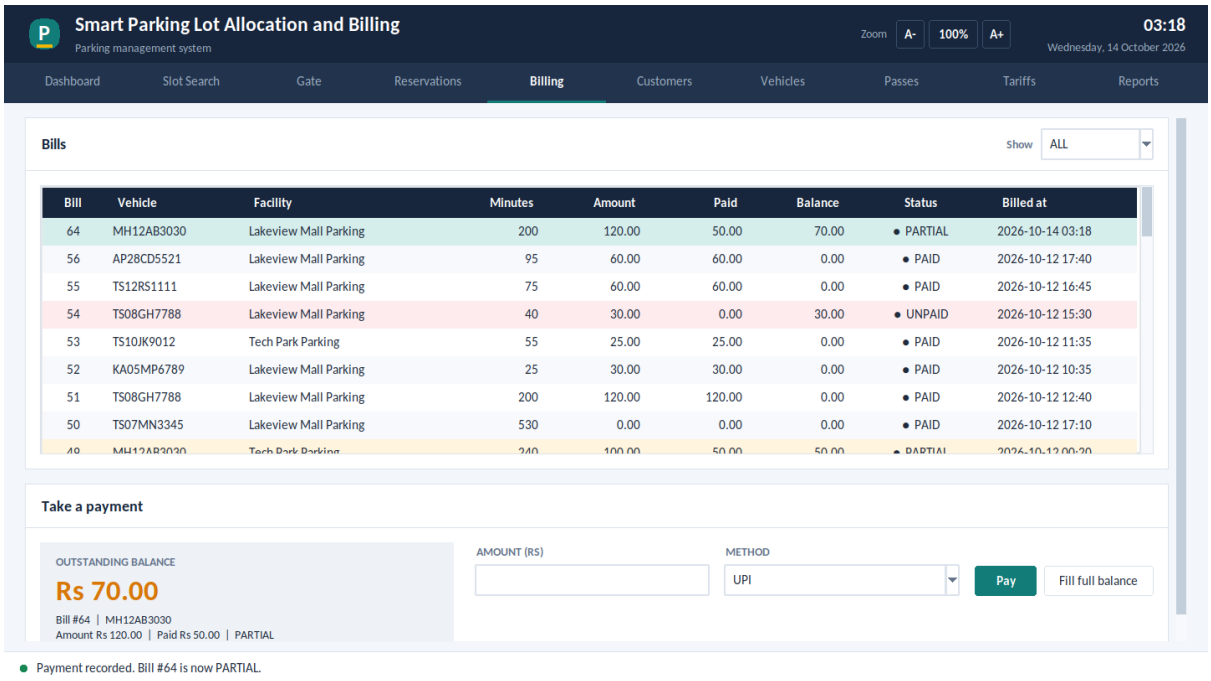


Figure 7. Reservation Management

The reservation screen demonstrates a booking and its assigned slot and time window.

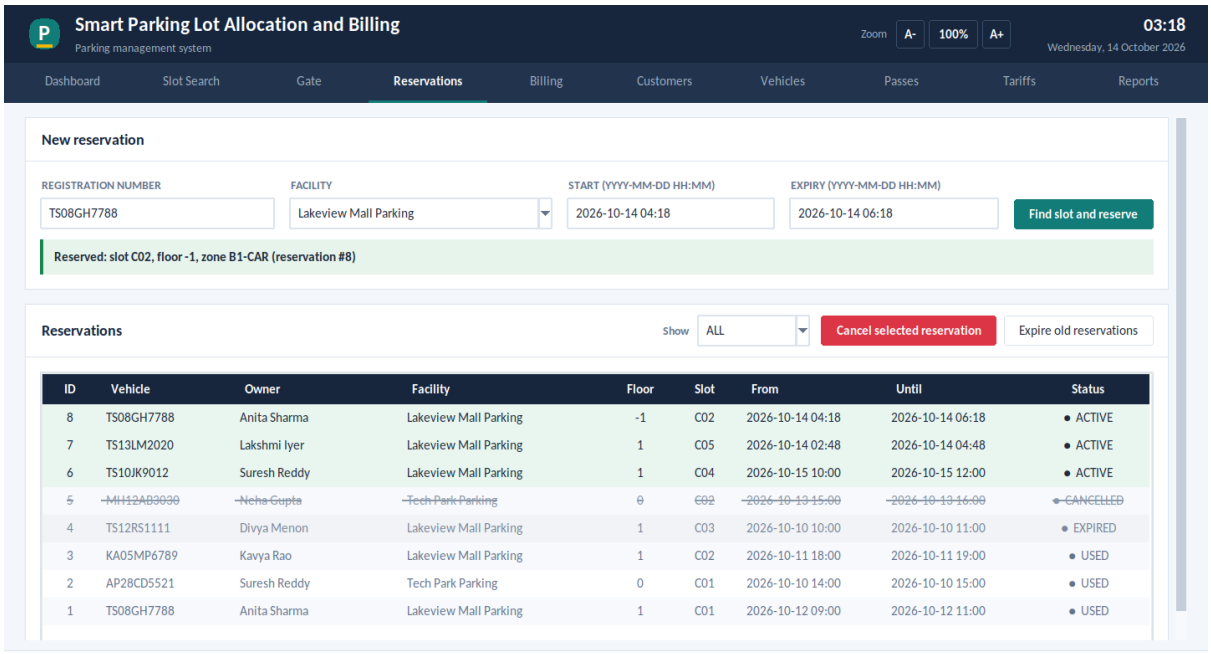


Figure 8. Customer Record Added

The customer screen shows a successfully added customer in the listing.

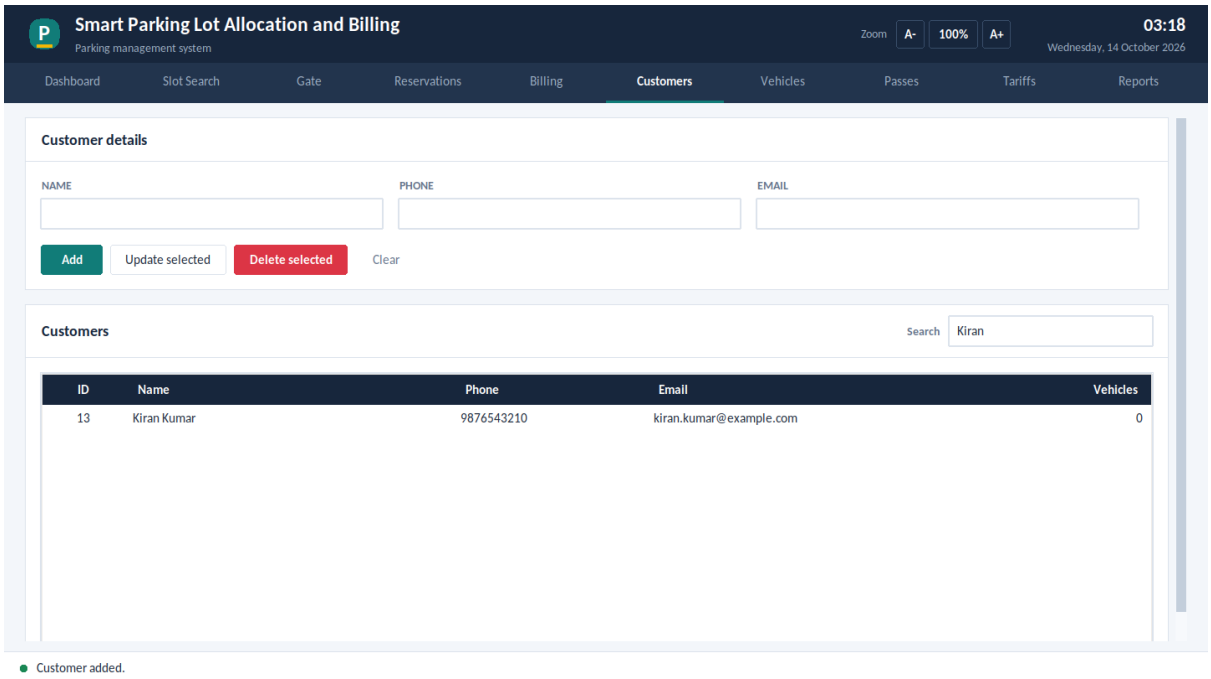


Figure 9. Peak-Hour Report

The report output summarizes entry volume by hour.

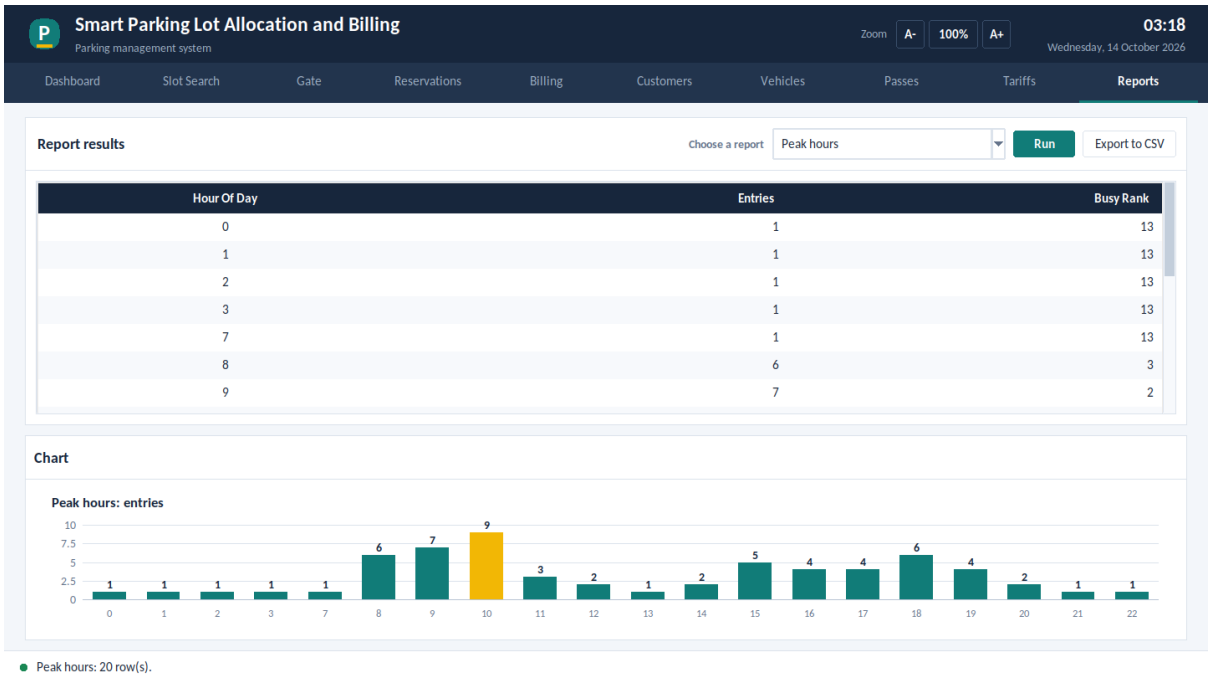


Figure 10. Violations Report

The violations report displays flagged parking or billing conditions.

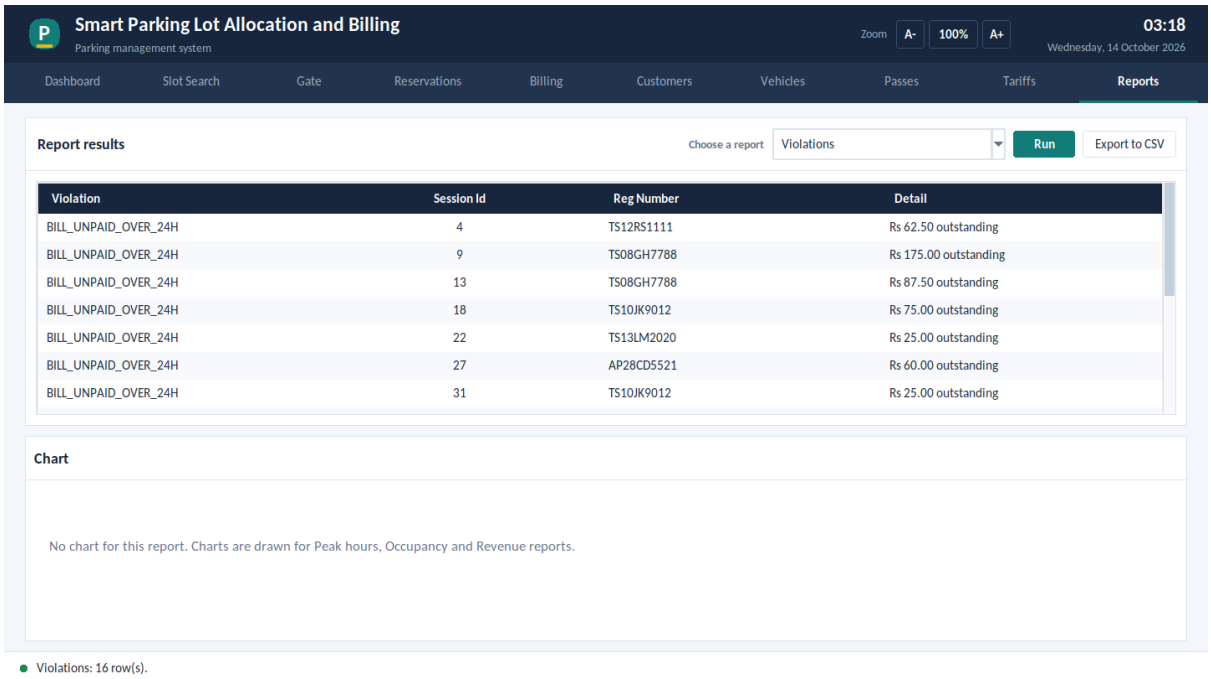


Figure 11. Revenue Report

The revenue report shows collected amounts using database-backed report queries.

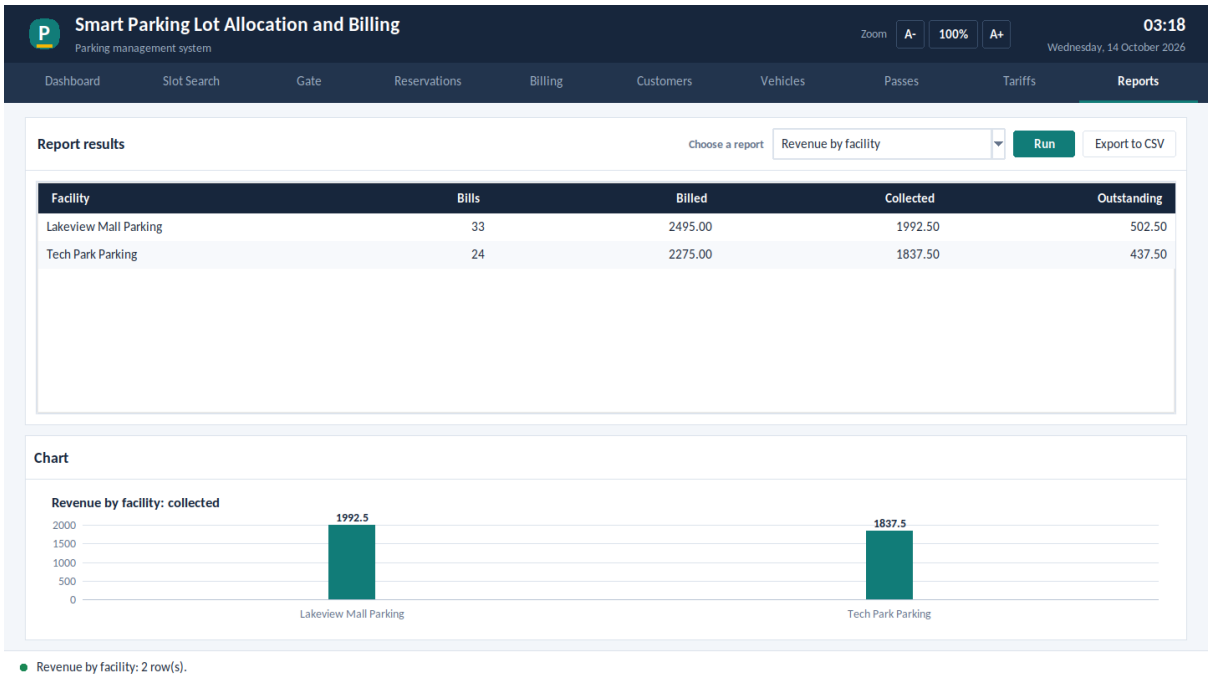
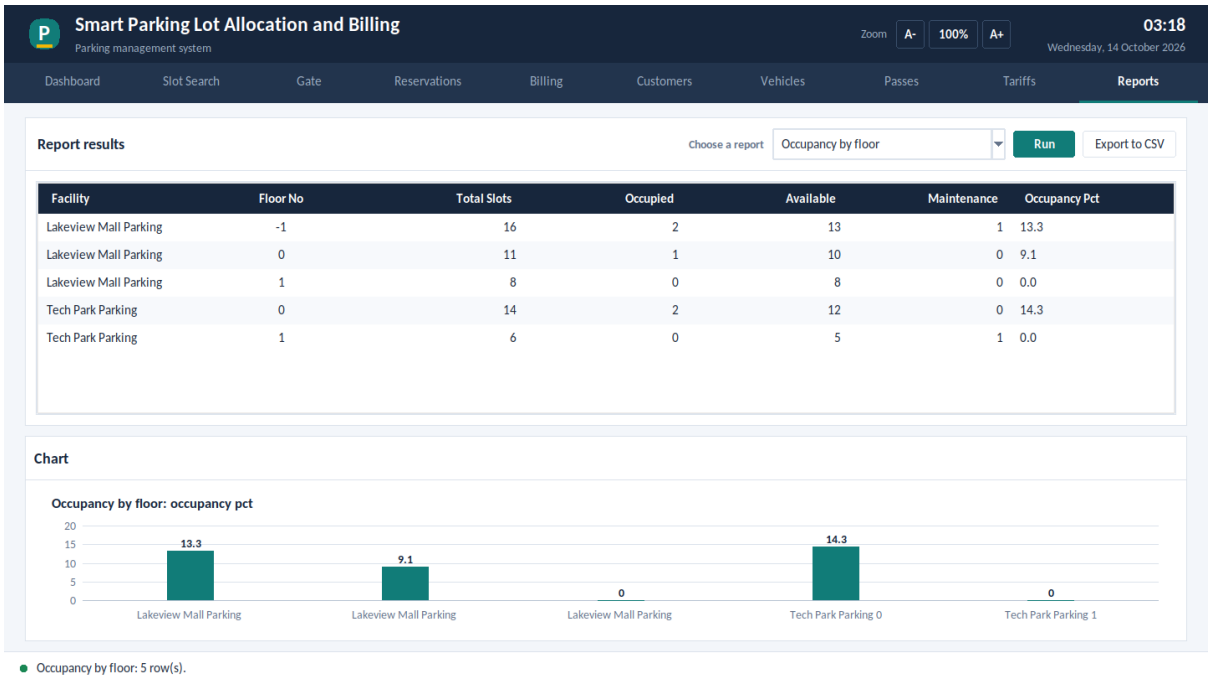


Figure 12. Occupancy Report

The occupancy report summarizes slot usage by facility and floor.



3.8 Testing and Results

The supplied test documentation reports 23 automated tests passing. It also documents 20 SQL statements in sql/05_constraint_tests.sql that are expected to fail because each deliberately violates a database rule. Manual screen tests are documented with screenshots.

The results below are based on the supplied docs/test_cases.md and docs/test_results.txt; they are not a claim that the database was independently rerun during report preparation.

Test Cases

Test ID	Feature/input	Expected result	Recorded result	Status
TC-01	Dashboard and slot search	Dashboard metrics load; car search returns only compatible car slots.	Documented automated coverage; suite output reports 23 tests passed.	PASS (documented)
TC-02	Slot status change	Available slot can be changed; occupied slot cannot be switched to maintenance.	Listed as passing in project test cases.	PASS (documented)
TC-03	Gate entry and duplicate entry	Entry allocates a slot; a second active entry for the same vehicle is refused.	Listed as passing in project test cases.	PASS (documented)
TC-04	Exit and duration billing	A 200-minute stay is charged using the applicable tariff and cap.	Automated test documentation records expected billing calculations.	PASS (documented)
TC-05	Part payment/overpayment	Part payment changes balance/status; overpayment is rejected; full payment marks the bill paid.	Listed as passing in project test cases.	PASS (documented)

TC-06	Reservation conflicts	Overlapping booking or invalid time window is rejected.	Listed as passing in project test cases.	PASS (documented)
TC-07	Customer validation	Invalid phone and duplicate phone are rejected.	Documented automated and manual screen tests.	PASS (documented)
TC-08	Unknown vehicle	Gate entry or reservation request for an unregistered vehicle is refused.	Manual screenshot and test cases document the error.	PASS (documented)
TC-09	Database constraints	Twenty deliberately invalid SQL statements are refused.	Project test notes state all 20 are refused with messages.	PASS (documented)
TC-10	Reports	All 10 application reports execute.	Listed as passing in project test cases.	PASS (documented)

The recorded test results indicate coverage of positive workflows and negative cases, including duplicate entry, invalid registration or phone number, reservation conflict, overpayment, and deletion of referenced customer data. The documented automated test run ends with “Ran 23 tests” and “OK.” The SQL constraint test file provides additional database-level negative tests.

These results are reported from the project’s supplied documentation rather than from an independent test run performed while preparing this report.

CHAPTER 4: CONCLUSION

The Smart Parking Lot Allocation and Billing System demonstrates how a relational database and desktop application can be integrated to manage parking operations. The design separates facility hierarchy, vehicle/customer data, reservations, passes, sessions, tariffs, bills, and payments into related tables. Keys and constraints preserve referential integrity, while triggers, functions, and stored procedures enforce important operational rules and calculate charges.

The Python application provides slot search, gate entry and exit, reservation management, customer and vehicle maintenance, pass and tariff management, billing, partial payments, and

reports. The supplied test artefacts report 23 automated tests passing and 20 database constraint tests rejecting invalid operations.

The project therefore provides a practical demonstration of database schema design, normalization, SQL programming, application connectivity, validation, CRUD operations, and reporting. It shows how a structured database can help improve the consistency and reliability of parking-management activities.

4.1 Limitations and Future Enhancements

Although the system supports the main parking-management workflows, several improvements could be considered for future versions.

1. Multi-user deployment: The project is configured as a local desktop application. A secure multi-user deployment with role-based access and centralized hosting could be added.
2. Payment gateway integration: External payment gateway integration and automated receipt delivery are not documented as implemented and could be considered future enhancements.
3. Audit logging: An audit trail could record important actions such as slot allocation, reservation cancellation, bill generation, and payment updates.
4. Automated backups: Scheduled database backups and a recovery process could improve data protection.
5. Advanced analytics: More detailed date-range filters and comparative revenue and occupancy reports could support better operational decisions.
6. Security improvements: Database credentials should be changed and managed securely before any non-academic deployment.

CHAPTER 5: REFERENCES

[1] Oracle, “MySQL 8.0 Reference Manual,” MySQL Documentation. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/. Accessed: October 15, 2026.

[2] Python Software Foundation, “Python 3 Documentation.” [Online]. Available: https://docs.python.org/3/. Accessed: October 15, 2026.

[3] Python Software Foundation, “tkinter — Python interface to Tcl/Tk,” Python Documentation. [Online]. Available: https://docs.python.org/3/library/tkinter.html. Accessed: October 15, 2026.

[4] Oracle, “MySQL Connector/Python Developer Guide.” [Online]. Available: https://dev.mysql.com/doc/connector-python/en/. Accessed: October 15, 2026.

CHAPTER 6: APPENDICES

APPENDIX A — COMPLETE DATABASE SCRIPT

The project archive contains the complete SQL implementation in the following files:

- sql/00_create_app_user.sql — creates the application database account.
- sql/01_schema.sql — creates the smart_parking database, 13 tables, constraints, indexes, triggers, functions, and stored procedures.
- sql/02_sample_data.sql — inserts demonstration data.
- sql/03_views.sql — creates nine reporting and operational views.
- sql/04_queries.sql — contains representative joins, subqueries, aggregations, window-function queries, and reporting queries.
- sql/05_constraint_tests.sql — contains 20 deliberate invalid-operation tests.
- sql/06_prototype_demo.sql — provides a scripted demonstration of entry, exit, billing, part payment, and occupancy.

These files are retained in the submitted project ZIP rather than duplicated in full in this report, keeping the main report readable and avoiding divergence between the report and executable SQL.

APPENDIX B — APPLICATION SOURCE CODE

The source code is organized under the app/ directory and launched using main.py.

The important modules are:

File	Purpose
main.py	Application entry point
app/gui.py	Tkinter screens and user interface
app/services.py	Business operations and service logic
app/db.py	Database access and connectivity
app/validators.py	Input validation
app/reports.py	Report definitions and database-backed reporting
app/theme.py	Interface styling
app/widgets.py	Reusable interface widgets

requirements.txt	Python dependency specification, including MySQL Connector/Python

Repository URL: https://github.com/SnigdhaChoudary/DBMS_PBL.git

APPENDIX C — ADDITIONAL TEST DATA AND OUTPUTS

Supplementary evidence is available in the following project files and folders:

- docs/test_cases.md
- docs/test_results.txt
- docs/console_demo_output.txt
- docs/user_guide.md
- screenshots/
- tests/
- sql/05_constraint_tests.sql

The screenshots included in Chapter 3 cover the dashboard, slot search, gate entry, billing, reservations, customer creation, and reports. The project also includes unit tests under tests/ and database constraint tests under sql/.